Job Board & Career Portal

Database-Backed Application – Final Project

Sadaf Kamal

Student Id: 0271799

Department of Business Administration, Monroe University, King Graduate School

CS630 - 154W: Database Systems

Professor Deepti Umesha

April 13, 2026

Summary

This project is about building a job board and career portal database that helps people find jobs and apply for them online. It supports three types of users: job seekers, employers, and administrators, and manages everything from posting jobs to submitting applications.

The database has 9 tables, all organized properly using Third Normal Form (3NF) to avoid duplication and keep data clean. It is created using SQLite and works on the Programiz online SQL compiler.

The main parts of the system include tables like Users, Employers, Job Seekers, Jobs, Applications, Interviews, Saved Jobs, Categories, and Reviews. These tables are connected using primary keys and foreign keys to maintain accurate and reliable data.

The project also includes:

- An **ER Diagram** to show how everything is connected
- **schema.sql** that will create the structure of database.
- **sample_data.sql** to help insert sample data.
- **reset.sql** to deletes database to get clean.

Overall, this project demonstrates how a complete job portal system can be designed and managed using a structured database.

Introduction

Project Background

Online job platforms like Indeed, LinkedIn, and Glassdoor handle a huge amount of data. They need to manage job listings, user profiles, applications, and interactions between job seekers and employers.

This project creates a simplified version of a job board and career portal to show how such systems work at the database level. It focuses on building a well-structured database using proper design techniques like normalization, relationships, and data integrity, along with practical SQL implementation.

Project Objectives

- Design a well-structured database (in 3NF) for a job portal
- Build the database using SQLite (compatible with Programiz)
- Add sample data for all 9 tables to show how the system works
- Clearly explain the design, relationships, and decisions made
- Include a reset script to easily delete and rebuild the database

Scope and Boundaries

The portal does the following:

- **Employers** can register, create a company profile, and post job openings with salary details and deadlines.
- **Job Seekers** can sign up, create profiles with their skills and education, search and save jobs, and apply with cover letters.
- The system tracks each application through these stages: **Pending → Reviewed → Shortlisted → Hired or Rejected.**
- If a job seeker is shortlisted, they can be scheduled for an **interview**, and the system tracks the interview's outcome.

- Job seekers can also leave **reviews and ratings** for employers they've worked with.

What's not included:

This database doesn't handle user logins, resume file storage, job search features, or notifications. These would be handled by the application itself.

Data Modeling

Entities and Attributes

Based on the core business processes of the job portal, the following nine entities were identified:

Table 1

Job Portal Database Entities

Entity	Primary Key	Key Attributes
Users	user_id (PK)	full_name, email, role, created_at
Employers	employer_id (PK)	company_name, industry, city, country
JobSeekers	seeker_id (PK)	skills, experience_years, education
Categories	category_id (PK)	category_name, description
Jobs	job_id (PK)	title, job_type, salary_min, salary_max, status
Applications	application_id (PK)	cover_letter, status, applied_at
Interviews	interview_id (PK)	scheduled_at, mode, outcome
SavedJobs	saved_id (PK)	saved_at
Reviews	review_id (PK)	rating (1–5), comment, reviewed_at

Entity Relationships

Users (1) → (1) Employer

One user account maps to exactly one employer profile.

Employers (1) → (M) Jobs

One employer can post many jobs, but a job belongs to only one employer.

JobSeekers (M) → (M) Jobs

Many job seekers can apply for many jobs (A job can have many applicants, and a seeker can apply for many jobs).

JobSeekers (M) → (M) Applications

Many job seekers can submit many applications, and each application can be tied to one job seeker.

Applications (1) → (1) Interviews

Each application can have one interview, but an interview relates to a specific application (one-to-one).

JobSeekers (1) → (M) Reviews

One job seeker can leave many reviews for employers, but each review is written by only one seeker.

Employers (1) → (M) Reviews

One employer can receive many reviews from different job seekers, but each review is tied to only one employer.

JobSeekers (M) → (M) SavedJobs

A job seeker can save many jobs, and each job can be saved by multiple seekers.

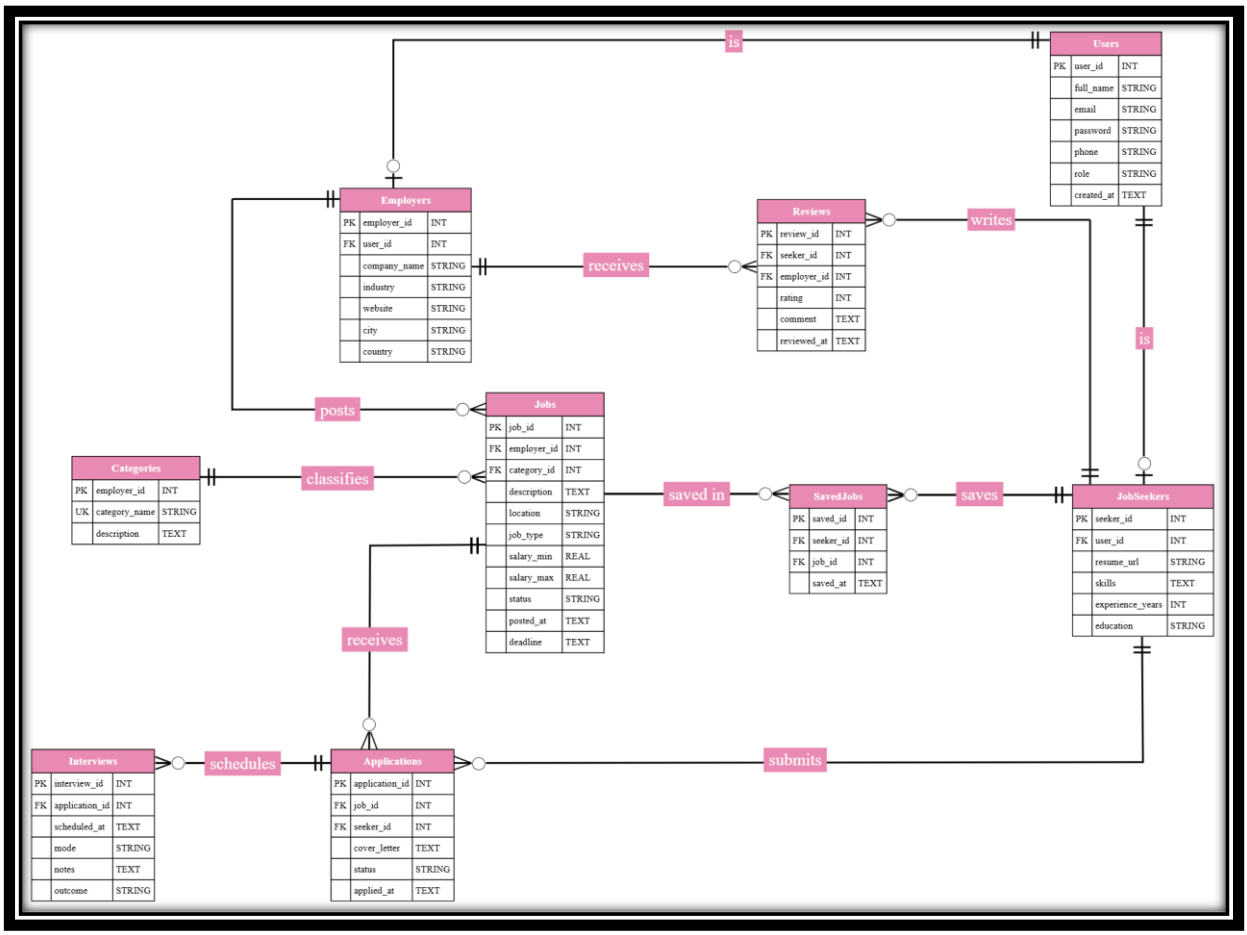
ER Diagram Description

The ER Diagram below shows the logical structure of the job portal database. Here's how the entities and relationships are organized:

- Entities are represented as boxes, and relationships are shown as lines connecting them. Crow's foot notation is used to indicate cardinality (one-to-one, one-to-many, many-to-many).
- Central Entity: The Jobs table sits at the center of the diagram and is connected to:
    - Employers (who post the jobs)
    - Categories (which classify the jobs)
    - Applications (which are submitted by job seekers)

- Users table is the parent for both Employers and JobSeekers using a 1:1 relationship. This means each user can either be an employer or a job seeker.
- SavedJobs and Reviews are junction/bridge tables, resolving the many-to-many (M:M) relationships:
    - ❖ SavedJobs links job seekers to the jobs they've saved.
    - ❖ Reviews connects job seekers with the employers they've interacted with.

ER Diagram



Legend for ER Diagram

- ***PK:*** Primary Key — Uniquely identifies each record in the table.
- ***FK:*** Foreign Key — A reference to a primary key in another table, establishing a relationship.
- ***UK:*** Unique Key — Ensures that all values in a column are unique across the table.

- ***Strings:*** Represent text-based data (e.g., names, addresses, descriptions).
- ***Integers:*** Represent numeric data (e.g., IDs, quantities).
- ***Real:*** Represents numeric data with decimal points (e.g., salaries).
- ***Relationship***

Design Decisions

User Role Architecture

Instead of creating separate tables for Job Seekers and Employers, I have a shared Users table. This table holds basic information like email, password, and role, while Employers and JobSeekers have their own extension tables that store role-specific data. This is called the Table-Per-Type (TPT) pattern.

Why this approach?

- It simplifies login logic since all credentials are stored in one place.
- It ensures each user has only one role-specific profile using a UNIQUE constraint on the user ID in both Employer and JobSeeker tables.
- It prevents duplication and makes it easy to add new roles (e.g., Agency Recruiter) in the future.

Normalization to 3NF

All tables are normalized to Third Normal Form (3NF):

- ***1NF:*** Every column has atomic values (no multi-value fields). For example, skills are stored in a single text field (comma-separated) for simplicity, but this would be split into a junction table in a full-scale system.
- ***2NF:*** Every non-key attribute depends on the entire primary key. We avoid partial dependencies because all primary keys are single-column surrogates.

- ***3NF:*** There are no transitive dependencies. For instance, employer city/country are stored in the Employers table, not in the Jobs table. This makes updates easy if an employer changes locations.

SQLite specific choices

This schema is designed for SQLite, used in Programiz's online compiler. Key differences from other databases like Oracle:

- AUTOINCREMENT is used for primary keys (Oracle uses SEQUENCE).
- DEFAULT (datetime('now')) is used to set timestamps (Oracle uses SYSDATE).
- CHECK constraints are used for fields like status and ratings to enforce valid values.

In SQLite, one needs to enable foreign key enforcement with PRAGMA foreign_keys = ON; (in Oracle, it’s automatic).

Application Status Lifecycle

The Applications table uses a CHECK constraint to enforce valid status values (e.g., Pending → Reviewed → Shortlisted → Hired or Rejected). This ensures consistent application tracking. Similar constraints are applied to Jobs.status and Interviews.outcome to keep values valid (e.g., Open / Closed / Paused for jobs, and Pass / Fail / Pending for interview outcomes).

Unique Constraints as Business Rules

Two key business rules are enforced with UNIQUE constraints:

- A job seeker can apply to a job only once: This is enforced by a UNIQUE(job_id, seeker_id) in the Applications table.
- A job seeker can only review an employer once: This is enforced by a UNIQUE(seeker_id, employer_id) in the Reviews table.

These rules ensure data integrity even if multiple applications or APIs interact with the database.

Salary as a Range

Instead of storing a single salary amount, the database stores salary_min and salary_max as separate values. This mirrors real-world job postings where salary is often a range. It also makes it easier to run range-based queries, like finding jobs with a maximum salary above $90,000.

Saved Jobs as an Explicit Junction Table

The many-to-many relationship between job seekers and saved jobs (bookmarked jobs) is represented by an explicit SavedJobs table. This avoids using arrays or JSON blobs and keeps the design relational. The table allows:

- Storing the timestamp when each job is saved.
- Running simple queries like "which jobs has seeker X saved?" or "how many seekers saved job Y?".

Database Schema – Table Description

Below are the details of each table in the database, including columns, data types, and constraints.

Users

The Users table holds all registered users, regardless of their role (JobSeeker, Employer, Admin). It stores login data, such as email and password.

Table 2

Users Table

Column	Type	Constraint	Description
user_id	INTEGER	PK, AUTOINCREMENT	Unique user identifier
full_name	TEXT	NOT NULL	User's full name
email	TEXT	NOT NULL, UNIQUE	Login email — must be unique
password	TEXT	NOT NULL	Hashed password string
phone	TEXT	nullable	Optional contact number
role	TEXT	CHECK IN (JobSeeker, Employer, Admin)	User Role (one of three roles)

Column	Type	Constraint	Description
created_at	TEXT	DEFAULT now()	Account creation timestamp

Employers

The Employers table stores information specific to employers, extending the Users table.

Table 3

Employers Table

Column	Type	Constraint	Description
employer_id	INTEGER	PK	Employer surrogate key
user_id	INTEGER	FK → Users, UNIQUE	Links to the Users table
company_name	TEXT	NOT NULL	Name of the hiring company
industry	TEXT	nullable	Industry/sector of the company
website	TEXT	nullable	Company website URL
city	TEXT	nullable	City of headquarters
country	TEXT	nullable	Country of headquarters

JobSeekers

The JobSeekers table stores information specific to job seekers, extending the Users table.

Table 4

JobSeekers Table

Column	Type	Constraint	Description
seeker_id	INTEGER	PK	Job Seeker surrogate key
user_id	INTEGER	FK → Users, UNIQUE	Links to the Users table
resume_url	TEXT	nullable	Link to hosted resume
skills	TEXT	nullable	Comma-separated skills list
experience_years	INTEGER	DEFAULT 0	Years of work experience
education	TEXT	nullable	Highest degree or institution

Categories

The Categories table stores job categories to avoid repetition in job listings.

Table 5

Categories Table

Column	Type	Constraint	Description
category_id	INTEGER	PK	Category surrogate key
category_name	TEXT	NOT NULL, UNIQUE	Display name of category
description	TEXT	nullable	Optional longer description

Jobs

The Jobs table stores information about job listings, referencing Employers and Categories.

Table 6

Jobs Table

Column	Type	Constraint	Description
job_id	INTEGER	PK	Job surrogate key
employer_id	INTEGER	FK → Employers	Employer who posted the job
category_id	INTEGER	FK → Categories	Job category
title	TEXT	NOT NULL	Job title / position name
description	TEXT	NOT NULL	Full job description
location	TEXT	nullable	City or Remote
job_type	TEXT	CHECK (4 values)	Full-Time / Part-Time / etc.
salary_min	REAL	nullable	Minimum salary in the range
salary_max	REAL	nullable	Maximum salary in the range
status	TEXT	CHECK, DEFAULT Open	Open / Closed / Paused
posted_at	TEXT	DEFAULT now()	When listing was created
deadline	TEXT	nullable	Application deadline date

Applications

The Applications table tracks job applications by job seekers, ensuring each seeker applies only once per job.

Table 7

Applications Table

Column	Type	Constraint	Description
application_id	INTEGER	PK	Application surrogate key

Column	Type	Constraint	Description
job_id	INTEGER	FK → Jobs	The job being applied to
seeker_id	INTEGER	FK → JobSeekers	The applying seeker
cover_letter	TEXT	nullable	Optional cover letter text
status	TEXT	CHECK (5 values)	Application status stage
applied_at	TEXT	DEFAULT now()	Submission timestamp

Interviews

The Interviews table tracks interviews related to job applications, ensuring one interview per application.

Table 8

Interview Table

Column	Type	Constraint	Description
interview_id	INTEGER	PK	Interview surrogate key
application_id	INTEGER	FK, UNIQUE	Links to Applications (1:1)
scheduled_at	TEXT	NOT NULL	Date and time of interview
mode	TEXT	CHECK (3 values)	In-Person / Phone / Video
notes	TEXT	nullable	Interviewer notes
outcome	TEXT	CHECK (4 values)	Pass / Fail / No Show / Pending

SavedJobs

The SavedJobs table is a junction table that resolves the many-to-many relationship between job seekers and saved jobs.

Table 9

SavedJobs Table

Column	Type	Constraint	Description
saved_id	INTEGER	PK	Surrogate key
seeker_id	INTEGER	FK → JobSeekers	The seeker who saved the job
job_id	INTEGER	FK → Jobs	The saved job listing
saved_at	TEXT	DEFAULT now()	When the bookmark was created

Reviews

The Reviews table allows job seekers to rate and review employers.

Table 10

Reviews Table

Column	Type	Constraint	Description
review_id	INTEGER	PK	Review surrogate key
seeker_id	INTEGER	FK → JobSeekers	Seeker who wrote the review
employer_id	INTEGER	FK → Employers	Employer being reviewed
rating	INTEGER	CHECK BETWEEN 1-5	Star rating out of 5
comment	TEXT	nullable	Written review text
reviewed_at	TEXT	DEFAULT now()	Timestamp of the review

SQL Implementation

These three SQL files are created for SQLite and tested using the Programiz SQL online compiler. Below is a description of each file:

SQL/schema.sql – DDL

This file defines the structure of the database by creating the tables with primary keys, foreign keys, check constraints, unique constraints, and default values. Tables are created in a dependency order (referencing tables are created first).

```
-- ================================================================
-- Job Board & Career Portal - Schema (SQLite / Programiz)
-- ================================================================
-- 1. Users (base table for all user types)
CREATE TABLE Users (
    user_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name    TEXT    NOT NULL,
    email        TEXT    NOT NULL UNIQUE,
    password     TEXT    NOT NULL,
    phone        TEXT,
    role         TEXT    NOT NULL CHECK(role IN ('JobSeeker','Employer','Admin')),
    created_at   TEXT    DEFAULT (datetime('now'))
);
```

```
-- 2. Employers (extended Users)
CREATE TABLE Employers (
    employer_id  INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id      INTEGER NOT NULL UNIQUE,
    company_name TEXT    NOT NULL,
    industry     TEXT,
    website      TEXT,
    city         TEXT,
    country      TEXT,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

-- 3. JobSeekers (extends Users)
CREATE TABLE JobSeekers (
    seeker_id   INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id     INTEGER NOT NULL UNIQUE,
    resume_url  TEXT,
    skills      TEXT,
    experience_years INTEGER DEFAULT 0,
    education   TEXT,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

-- 4. Categories
CREATE TABLE Categories (
    category_id   INTEGER PRIMARY KEY AUTOINCREMENT,
    category_name TEXT NOT NULL UNIQUE,
    description   TEXT
);

-- 5. Jobs
CREATE TABLE Jobs (
    job_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    employer_id  INTEGER NOT NULL,
    category_id  INTEGER NOT NULL,
    title        TEXT    NOT NULL,
```

```
    description  TEXT    NOT NULL,
    location     TEXT,
    job_type     TEXT    CHECK(job_type IN ('Full-Time','Part-Time','Contract','Internship')),
    salary_min   REAL,
    salary_max   REAL,
    status       TEXT    DEFAULT 'Open' CHECK(status IN ('Open','Closed','Paused')),
    posted_at    TEXT    DEFAULT (datetime('now')),
    deadline     TEXT,
    FOREIGN KEY (employer_id) REFERENCES Employers(employer_id),
    FOREIGN KEY (category_id) REFERENCES Categories(category_id)
);
```

-- 6. Applications

```
CREATE TABLE Applications (
    application_id INTEGER PRIMARY KEY AUTOINCREMENT,
    job_id         INTEGER NOT NULL,
    seeker_id      INTEGER NOT NULL,
    cover_letter   TEXT,
    status         TEXT DEFAULT 'Pending' CHECK(status IN
                        ('Pending','Reviewed','Shortlisted','Rejected','Hired')),
    applied_at     TEXT DEFAULT (datetime('now')),
                        UNIQUE(job_id, seeker_id),
    FOREIGN KEY (job_id)     REFERENCES Jobs(job_id),
    FOREIGN KEY (seeker_id)  REFERENCES JobSeekers(seeker_id)
);
```

-- 7. Interviews

```
CREATE TABLE Interviews (
    interview_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    application_id  INTEGER NOT NULL UNIQUE,
    scheduled_at    TEXT    NOT NULL,
    mode            TEXT    CHECK(mode IN ('In-Person','Phone','Video')),
    notes           TEXT,
    outcome         TEXT    CHECK(outcome IN ('Pass','Fail','No Show','Pending'))
                            DEFAULT 'Pending',
```

```
    FOREIGN KEY (application_id) REFERENCES Applications(application_id)
);
```

-- 8. SavedJobs (bookmarks)

```
CREATE TABLE SavedJobs (
    saved_id   INTEGER PRIMARY KEY AUTOINCREMENT,
    seeker_id  INTEGER NOT NULL,
    job_id     INTEGER NOT NULL,
    saved_at   TEXT DEFAULT (datetime('now')),
                        UNIQUE(seeker_id, job_id),
    FOREIGN KEY (seeker_id) REFERENCES JobSeekers(seeker_id),
    FOREIGN KEY (job_id)    REFERENCES Jobs(job_id)
);
```

-- 9. Reviews (seeker reviews employer)

```
CREATE TABLE Reviews (
    review_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    seeker_id     INTEGER NOT NULL,
    employer_id INTEGER NOT NULL,
    rating          INTEGER CHECK(rating BETWEEN 1 AND 5),
    comment     TEXT,
    reviewed_at TEXT DEFAULT (datetime('now')),
                        UNIQUE(seeker_id, employer_id),
    FOREIGN KEY (seeker_id)   REFERENCES JobSeekers(seeker_id),
    FOREIGN KEY (employer_id) REFERENCES Employers(employer_id)
);
```

Available Tables

Applications

application_id	job_id	seeker_id	cover_letter	status	applied_at
empty					

Categories

category_id	category_name	description
empty		

Employers

employer_id	user_id	company_name	industry	website	city	country
empty						

Interviews

interview_id	application_id	scheduled_at	mode	notes	outcome
empty					

Jobs

job_id	employer_id	category_id	title	description	location	job_type	salary_min	salary_max	status	posted_at	deadline
empty											

JobSeekers

seeker_id	user_id	resume_url	skills	experience_years	education
empty					

Reviews

review_id	seeker_id	employer_id	rating	comment	reviewed_at
empty					

SavedJobs

saved_id	seeker_id	job_id	saved_at
empty			

Users

user_id	full_name	email	password	phone	role	created_at
empty						

SQL/sample_data.sql – DML

```
-- ==================================================================
-- Job Board & Career Portal - Sample Data (SQLite / Programiz)
-- ==================================================================
-- Users
INSERT INTO Users (full_name, email, password, phone, role) VALUES
('Alice Johnson',  'alice@email.com',  'hashed_pw_1', '555-0101', 'JobSeeker'),
('Bob Martinez',  'bob@email.com',  'hashed_pw_2', '555-0102', 'JobSeeker'),
('Carol White',   'carol@email.com',  'hashed_pw_3', '555-0103', 'JobSeeker'),
('David Kim',  'david@email.com',    'hashed_pw_4', '555-0104', 'JobSeeker'),
('Eva Patel',  'eva@email.com',  'hashed_pw_5', '555-0105', 'JobSeeker'),
('TechCorp Inc',    'hr@techcorp.com',  'hashed_pw_6', '555-0201', 'Employer'),
('FinancePro LLC',  'jobs@financepro.com','hashed_pw_7', '555-0202', 'Employer'),
('HealthPlus',  'recruit@hplus.com', 'hashed_pw_8', '555-0203', 'Employer'),
('Admin User',  'admin@portal.com', 'hashed_pw_9', NULL, 'Admin');
```

Users

user_id	full_name	email	password	phone	role	created_at
1	Alice Johnson	alice@email.com	hashed_pw_1	555-0101	JobSeeker	2026-04-02 22:38:53
2	Bob Martinez	bob@email.com	hashed_pw_2	555-0102	JobSeeker	2026-04-02 22:38:53
3	Carol White	carol@email.com	hashed_pw_3	555-0103	JobSeeker	2026-04-02 22:38:53
4	David Kim	david@email.com	hashed_pw_4	555-0104	JobSeeker	2026-04-02 22:38:53
5	Eva Patel	eva@email.com	hashed_pw_5	555-0105	JobSeeker	2026-04-02 22:38:53
6	TechCorp Inc	hr@techcorp.com	hashed_pw_6	555-0201	Employer	2026-04-02 22:38:53
7	FinancePro LLC	jobs@financepro.com	hashed_pw_7	555-0202	Employer	2026-04-02 22:38:53
8	HealthPlus	recruit@hplus.com	hashed_pw_8	555-0203	Employer	2026-04-02 22:38:53
9	Admin User	admin@portal.com	hashed_pw_9		Admin	2026-04-02 22:38:53

```
-- Employers
INSERT INTO Employers (user_id, company_name, industry, website, city, country)
VALUES
```

(6, 'TechCorp Inc', 'Technology', 'https://techcorp.com', 'New York', 'USA'),

(7, 'FinancePro LLC', 'Finance', 'https://financepro.com', 'Chicago', 'USA'),

(8, 'HealthPlus', 'Healthcare', 'https://healthplus.com', 'Boston', 'USA');

Employers

employer_id	user_id	company_name	industry	website	city	country
1	6	TechCorp Inc	Technology	https://techcorp.com	New York	USA
2	7	FinancePro LLC	Finance	https://financepro.com	Chicago	USA
3	8	HealthPlus	Healthcare	https://healthplus.com	Boston	USA

-- JobSeekers

INSERT INTO JobSeekers (user_id, skills, experience_years, education, resume_url)

VALUES

(1, 'Python, SQL, Django', 3, 'B.Sc Computer Science', 'https://resumes.io/alice'),

(2, 'Java, Spring Boot, AWS', 5, 'B.Sc Software Eng', 'https://resumes.io/bob'),

(3, 'Excel, Power BI, Accounting', 4, 'B.Sc Finance', 'https://resumes.io/carol'),

(4, 'Nursing, Patient Care, EMR', 2, 'B.S Nursing', 'https://resumes.io/david'),

(5, 'React, Node.js, TypeScript', 1, 'B.Sc IT', 'https://resumes.io/eva');

JobSeekers

seeker_id	user_id	resume_url	skills	experience_years	education
1	1	https://resumes.io/alice	Python, SQL, Django	3	B.Sc Computer Science
2	2	https://resumes.io/bob	Java, Spring Boot, AWS	5	B.Sc Software Eng
3	3	https://resumes.io/carol	Excel, Power BI, Accounting	4	B.Sc Finance
4	4	https://resumes.io/david	Nursing, Patient Care, EMR	2	B.S Nursing
5	5	https://resumes.io/eva	React, Node.js, TypeScript	1	B.Sc IT

-- Categories

INSERT INTO Categories (category_name, description)

VALUES

('Software Engineering', 'Development, architecture, and engineering roles'),

('Finance & Accounting', 'Financial analysis, accounting, and auditing roles'),

('Healthcare', 'Medical, nursing, and clinical roles'),

('Data Science', 'Analytics, ML, and data engineering roles'),

('Marketing', 'Digital marketing and brand management');

Categories

category_id	category_name	description
1	Software Engineering	Development, architecture, and engineering roles
2	Finance & Accounting	Financial analysis, accounting, and auditing roles
3	Healthcare	Medical, nursing, and clinical roles
4	Data Science	Analytics, ML, and data engineering roles
5	Marketing	Digital marketing and brand management

-- Jobs

INSERT INTO Jobs (employer_id, category_id, title, description, location, job_type, salary_min, salary_max, status, deadline) VALUES

(1, 1, 'Backend Python Developer', 'Build and maintain APIs using Django and PostgreSQL.', 'New York', 'Full-Time', 85000, 110000, 'Open', '2025-06-30'),
(1, 4, 'Data Analyst', 'Analyze product data, build dashboards, write SQL reports.', 'Remote', 'Full-Time', 70000, 95000, 'Open', '2025-07-15'),
(1, 1, 'Frontend React Developer', 'Build interactive UIs with React and TypeScript.', 'New York', 'Contract', 60000, 80000, 'Open', '2025-06-20'),
(2, 2, 'Financial Analyst', 'Prepare financial reports and forecasting models.', 'Chicago', 'Full-Time', 75000, 100000, 'Open', '2025-07-01'),
(2, 2, 'Accounting Intern', 'Assist accounting team with bookkeeping and reconciliation.', 'Chicago', 'Internship', 20000, 30000, 'Closed', '2025-05-01'),
(3, 3, 'Registered Nurse', 'Provide patient care in our inpatient medical ward.', 'Boston', 'Full-Time', 65000, 85000, 'Open', '2025-07-31'),
(3, 3, 'Medical Records Specialist', 'Maintain accurate patient records using EMR systems.', 'Boston', 'Part-Time', 35000, 45000, 'Open', '2025-08-01');

Jobs

job_id	employer_id	category_id	title	description	location	job_type	salary_min	salary_max	status	posted_at	deadline
1	1	1	Backend Python Developer	Build and maintain APIs using Django and PostgreSQL.	New York	Full-Time	85000	110000	Open	2026-04-02 22:38:53	2025-06-30
2	1	4	Data Analyst	Analyze product data, build dashboards, write SQL reports.	Remote	Full-Time	70000	95000	Open	2026-04-02 22:38:53	2025-07-15
3	1	1	Frontend React Developer	Build interactive UIs with React and TypeScript.	New York	Contract	60000	80000	Open	2026-04-02 22:38:53	2025-06-20
4	2	2	Financial Analyst	Prepare financial reports and forecasting models.	Chicago	Full-Time	75000	100000	Open	2026-04-02 22:38:53	2025-07-01
5	2	2	Accounting Intern	Assist accounting team with bookkeeping and reconciliation.	Chicago	Internship	20000	30000	Closed	2026-04-02 22:38:53	2025-05-01
6	3	3	Registered Nurse	Provide patient care in our inpatient medical ward.	Boston	Full-Time	65000	85000	Open	2026-04-02 22:38:53	2025-07-31
7	3	3	Medical Records Specialist	Maintain accurate patient records using EMR systems.	Boston	Part-Time	35000	45000	Open	2026-04-02 22:38:53	2025-08-01

-- Applications

INSERT INTO Applications (job_id, seeker_id, cover_letter, status) VALUES
(1, 1, 'I have 3 years of Django experience and would love to join TechCorp.', 'Shortlisted'),
(1, 5, 'Eager to transition into backend, strong Python fundamentals.', 'Pending'),
(2, 1, 'Experienced with SQL and dashboards, ready to dive into analytics.', 'Reviewed'),
(3, 5, 'React and TypeScript are my primary stack. Portfolio attached.', 'Shortlisted'),

(4, 3, 'Four years in financial analysis, CFA Level 1 candidate.', 'Hired'),

(5, 3, 'Would like to gain more experience in accounting practices.', 'Rejected'),

(6, 4, 'Registered nurse with 2 years ICU experience.', 'Shortlisted'),

(7, 4, 'Proficient with Epic EMR and medical record management.', 'Pending');

Applications

application_id	job_id	seeker_id	cover_letter	status	applied_at
1	1	1	I have 3 years of Django experience and would love to join TechCorp.	Shortlisted	2026-04-02 22:38:53
2	1	5	Eager to transition into backend, strong Python fundamentals.	Pending	2026-04-02 22:38:53
3	2	1	Experienced with SQL and dashboards, ready to dive into analytics.	Reviewed	2026-04-02 22:38:53
4	3	5	React and TypeScript are my primary stack. Portfolio attached.	Shortlisted	2026-04-02 22:38:53
5	4	3	Four years in financial analysis, CFA Level 1 candidate.	Hired	2026-04-02 22:38:53
6	5	3	Would like to gain more experience in accounting practices.	Rejected	2026-04-02 22:38:53
7	6	4	Registered nurse with 2 years ICU experience.	Shortlisted	2026-04-02 22:38:53
8	7	4	Proficient with Epic EMR and medical record management.	Pending	2026-04-02 22:38:53

-- Interviews

INSERT INTO Interviews (application_id, scheduled_at, mode, outcome) VALUES

(1, '2025-05-20 10:00:00', 'Video', 'Pass'),

(4, '2025-05-22 14:00:00', 'Video', 'Pass'),

(5, '2025-05-15 09:00:00', 'In-Person', 'Pass'),

(7, '2025-05-25 11:00:00', 'Phone', 'Pending');

Interviews

interview_id	application_id	scheduled_at	mode	notes	outcome
1	1	2025-05-20 10:00:00	Video		Pass
2	4	2025-05-22 14:00:00	Video		Pass
3	5	2025-05-15 09:00:00	In-Person		Pass
4	7	2025-05-25 11:00:00	Phone		Pending

-- SavedJobs

INSERT INTO SavedJobs (seeker_id, job_id) VALUES

(1, 4),

(2, 1),

(2, 2),

(3, 4),

(5, 1),

(5, 2);

SavedJobs

saved_id	seeker_id	job_id	saved_at
1	1	4	2026-04-02 22:38:53
2	2	1	2026-04-02 22:38:53
3	2	2	2026-04-02 22:38:53
4	3	4	2026-04-02 22:38:53
5	5	1	2026-04-02 22:38:53
6	5	2	2026-04-02 22:38:53

-- Reviews

INSERT INTO Reviews (seeker_id, employer_id, rating, comment) VALUES

(3, 2, 5, 'Great onboarding and professional team at FinancePro.'),

(1, 1, 4, 'TechCorp has a strong engineering culture and fair interview process.'),

(4, 3, 4, 'HealthPlus was responsive and the hiring process was smooth.');

Reviews

review_id	seeker_id	employer_id	rating	comment	reviewed_at
1	3	2	5	Great onboarding and professional team at FinancePro.	2026-04-02 22:38:53
2	1	1	4	TechCorp has a strong engineering culture and fair interview process.	2026-04-02 22:38:53
3	4	3	4	HealthPlus was responsive and the hiring process was smooth.	2026-04-02 22:38:53

SQL/reset.sql – Drop Script

This file deletes (drops) all tables in reverse order to respect foreign key constraints.

You can run this script to wipe and rebuild the schema.

-- ==========
-- Job Board & Career Portal – Reset.SQL (SQLite / Programiz)
-- ==========
-- Drop tables in reverse FK dependency order

DROP TABLE IF EXISTS Reviews;

DROP TABLE IF EXISTS SavedJobs;

DROP TABLE IF EXISTS Interviews;

DROP TABLE IF EXISTS Applications;

DROP TABLE IF EXISTS Jobs;

DROP TABLE IF EXISTS Categories;

DROP TABLE IF EXISTS JobSeekers;

DROP TABLE IF EXISTS Employers;

DROP TABLE IF EXISTS Users;

Programiz
Online SQL Editor

Input | Run SQL | Available Tables

```
-- ==========
-- Job Board & Career Portal – Reset.SQL (SQLite / Programiz)
-- ==========
-- Drop tables in reverse FK dependency order
DROP TABLE IF EXISTS Reviews;
DROP TABLE IF EXISTS SavedJobs;
DROP TABLE IF EXISTS Interviews;
DROP TABLE IF EXISTS Applications;
DROP TABLE IF EXISTS Jobs;
DROP TABLE IF EXISTS Categories;
DROP TABLE IF EXISTS JobSeekers;
DROP TABLE IF EXISTS Employers;
DROP TABLE IF EXISTS Users;
```

Output

SQL query successfully executed. However, the result set is empty.

Sample Queries

The following queries demonstrate how to interact with the database using SQLite on Programiz.

Query 1: All Open Jobs with Employer and Category

This query retrieves a list of open jobs, along with the employer's name and job category.

```
SELECT j.title, e.company_name, c.category_name,
      j.job_type, j.salary_min, j.salary_max, j.location
FROM Jobs j
JOIN Employers e  ON j.employer_id = e.employer_id
JOIN Categories c ON j.category_id = c.category_id
WHERE j.status = 'Open'
ORDER BY j.posted_at DESC;
```

Input

Run SQL

```
--Query 1: All Open Jobs with Employer and Category
--This query retrieves a list of open jobs, along with the employer's name and job category.
SELECT j.title, e.company_name, c.category_name,
       j.job_type, j.salary_min, j.salary_max, j.location
FROM Jobs j
JOIN Employers e  ON j.employer_id = e.employer_id
JOIN Categories c ON j.category_id = c.category_id
WHERE j.status = 'Open'
ORDER BY j.posted_at DESC;
```

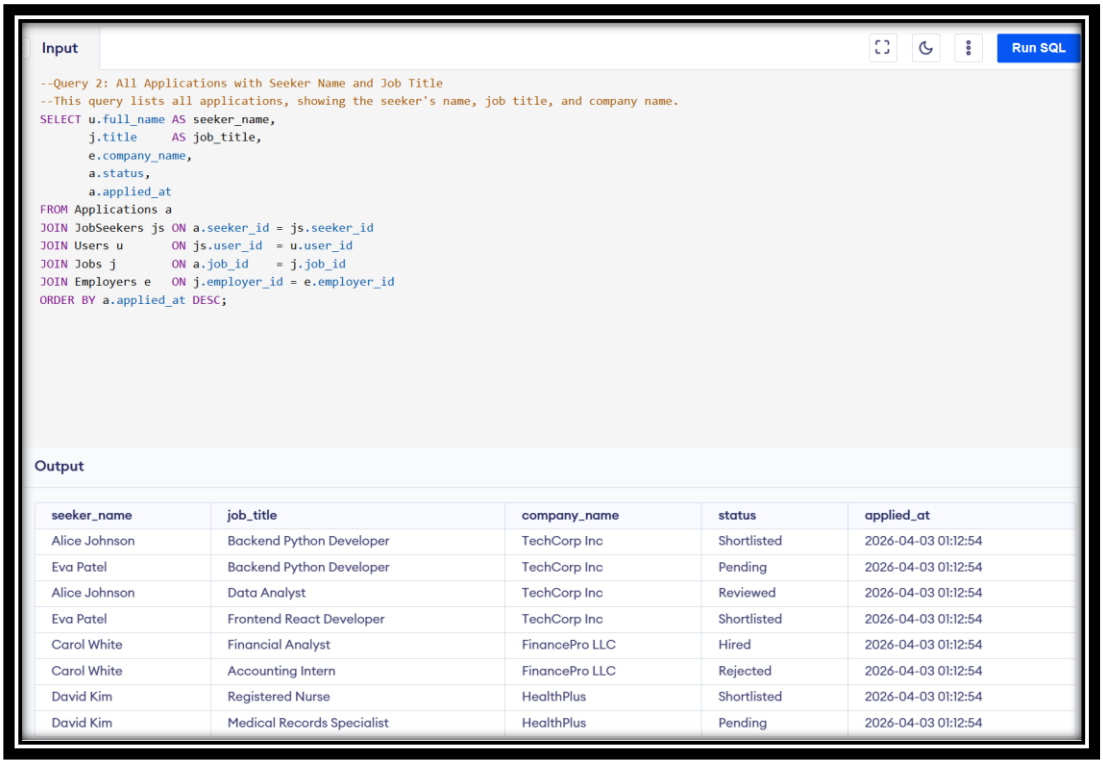
Output

title	company_name	category_name	job_type	salary_min	salary_max	location
Backend Python Developer	TechCorp Inc	Software Engineering	Full-Time	85000	110000	New York
Data Analyst	TechCorp Inc	Data Science	Full-Time	70000	95000	Remote
Frontend React Developer	TechCorp Inc	Software Engineering	Contract	60000	80000	New York
Financial Analyst	FinancePro LLC	Finance & Accounting	Full-Time	75000	100000	Chicago
Registered Nurse	HealthPlus	Healthcare	Full-Time	65000	85000	Boston
Medical Records Specialist	HealthPlus	Healthcare	Part-Time	35000	45000	Boston

Query 2: All Applications with Seeker Name and Job Title

This query lists all applications, showing the seeker's name, job title, and company name.

```
SELECT u.full_name AS seeker_name,
       j.title     AS job_title,
       e.company_name,
       a.status,
       a.applied_at
FROM Applications a
JOIN JobSeekers js ON a.seeker_id = js.seeker_id
JOIN Users u       ON js.user_id  = u.user_id
JOIN Jobs j        ON a.job_id    = j.job_id
JOIN Employers e   ON j.employer_id = e.employer_id
ORDER BY a.applied_at DESC;
```



seeker_name	job_title	company_name	status	applied_at
Alice Johnson	Backend Python Developer	TechCorp Inc	Shortlisted	2026-04-03 01:12:54
Eva Patel	Backend Python Developer	TechCorp Inc	Pending	2026-04-03 01:12:54
Alice Johnson	Data Analyst	TechCorp Inc	Reviewed	2026-04-03 01:12:54
Eva Patel	Frontend React Developer	TechCorp Inc	Shortlisted	2026-04-03 01:12:54
Carol White	Financial Analyst	FinancePro LLC	Hired	2026-04-03 01:12:54
Carol White	Accounting Intern	FinancePro LLC	Rejected	2026-04-03 01:12:54
David Kim	Registered Nurse	HealthPlus	Shortlisted	2026-04-03 01:12:54
David Kim	Medical Records Specialist	HealthPlus	Pending	2026-04-03 01:12:54

Query 3: Shortlisted Applicants with Interview Details

This query shows shortlisted applicants with interview details like the interview date, mode, and outcome.

```
SELECT u.full_name, j.title, i.scheduled_at, i.mode, i.outcome
FROM Interviews i
JOIN Applications a  ON i.application_id = a.application_id
JOIN JobSeekers js   ON a.seeker_id = js.seeker_id
JOIN Users u         ON js.user_id  = u.user_id
```

```
JOIN Jobs j          ON a.job_id    = j.job_id
WHERE a.status = 'Shortlisted';
```

Input

```
--Query 3: Shortlisted Applicants with Interview Details
--This query shows shortlisted applicants with interview details like the interview date, mode, and outcome.
SELECT u.full_name, j.title, i.scheduled_at, i.mode, i.outcome
FROM Interviews i
JOIN Applications a  ON i.application_id = a.application_id
JOIN JobSeekers js   ON a.seeker_id = js.seeker_id
JOIN Users u         ON js.user_id  = u.user_id
JOIN Jobs j          ON a.job_id    = j.job_id
WHERE a.status = 'Shortlisted';
```

Output

full_name	title	scheduled_at	mode	outcome
Alice Johnson	Backend Python Developer	2025-05-20 10:00:00	Video	Pass
Eva Patel	Frontend React Developer	2025-05-22 14:00:00	Video	Pass
David Kim	Registered Nurse	2025-05-25 11:00:00	Phone	Pending

Query 4: Average Employer Rating with Review Count

This query calculates the average rating for each employer, along with the total number of reviews.

```
SELECT e.company_name,
      ROUND(AVG(r.rating), 2) AS avg_rating,
      COUNT(r.review_id)      AS total_reviews
FROM Reviews r
JOIN Employers e ON r.employer_id = e.employer_id
GROUP BY e.company_name
ORDER BY avg_rating DESC;
```

Input

```
--Query 4: Average Employer Rating with Review Count
--This query calculates the average rating for each employer, along with the total number of reviews.
SELECT e.company_name,
       ROUND(AVG(r.rating), 2) AS avg_rating,
       COUNT(r.review_id)      AS total_reviews
FROM Reviews r
JOIN Employers e ON r.employer_id = e.employer_id
GROUP BY e.company_name
ORDER BY avg_rating DESC;
```

Output

company_name	avg_rating	total_reviews
FinancePro LLC	5	1
TechCorp Inc	4	1
HealthPlus	4	1

Query 5: Jobs Saved by a Specific Seeker

This query shows all the jobs saved by a specific job seeker (e.g., Bob Martinez).

```
SELECT u.full_name AS seeker,
       j.title, e.company_name, sj.saved_at
FROM SavedJobs sj
JOIN JobSeekers js ON sj.seeker_id = js.seeker_id
JOIN Users u       ON js.user_id   = u.user_id
JOIN Jobs j        ON sj.job_id    = j.job_id
JOIN Employers e   ON j.employer_id = e.employer_id
WHERE u.full_name = 'Bob Martinez';
```

These queries demonstrate how to retrieve useful data from the job portal database, such as job listings, applications, interviews, reviews, and saved jobs.

GitHub Repository

The project is hosted in a public GitHub repository that provides access to all the source code, schema designs, sample data, and documentation. The repository follows a structured development process with incremental commits that document the progress from initial database schema design to the final implementation. The repository serves as a version-controlled space where the evolution of the project can be tracked, and collaboration can be facilitated if needed.

Repository Structure

The repository is organized as follows:

- ***schema.sql*** **—** DDL statements to create all nine tables
- ***sample_data.sql*** **—** INSERT statements with representative data
- ***Reset.sql*** **—** DROP statements to reset the database
- ***ER Diagram.pdf*** **—** Entity Relationship diagram
- ***CS630-154W - Job Board & Career Portal...pdf*** — Full project report
- ***README.md*** **—** Repository documentation and setup guide

Commit History

The GitHub repository includes multiple commits that document the progression of the project, covering key stages such as:

- The creation of the initial schema
- Refining database normalization and structure
- Adding relationships and constraints
- Populating the database with sample data
- Final testing and project documentation

Repository URL

The project is available at: **https://github.com/sadafkamal299-maker/job-board-career-portal-dB**

This GitHub repository provides access to the project and tracks its development. It's a useful resource for understanding or contributing to the job board and career portal database system.